

Inigo Quilez :: articles :: clever normalization of a mesh - 2004



Wait... an article about how to calculate vertex normals for a mesh? This is very easy right? Yes, but let's say you want high quality normals at faster speed than usual implementations, with a more compact code too. Ok, then this very short article is for you! I invented this little method around 2004 and I've now seen used in many places, which makes me happy! Let's have a look:

Getting rid of the divisions

Most mesh normalization implementations out there use the typical normal averaging of face normals to calculate the vertex normal. For that, one iterates all the faces on which the current vertex is contained, accumulate the normals and then divide by the amount of faces used in the accumulation.

Now think what this division is doing to the normal. It does not change its direction for sure, since a division by a scalar only affects the length of the normal. Actually we do not care about this length at all, since we will most probably normalize it to unit length. This means that right after the face normal accumulation, we are done, we don't need any division. Thus we can skip not only this operation but also all the code to keep track of the amount of faces that affect each vertex.

Getting rid of the normalizations (square roots and divisions)

Now think again what we are really doing when accumulating the face normals into vertex normals. We are making a linear combination of normals, with equal importance or weight for all of them. Is this good? Well, one would think that polygons with bigger area should probably contribute more to the final result. This indeed gives better quality vertex normals. So, let's try to calculate the area of each polygon... Hey, but wait! Open your primary school math book. Have a look to the definition of cross product, specially to the length of the cross product. Cheeses, the length of a cross product is proportional to the area of the parallelepiped created by the two vectors involved in the product. And is not our triangle normal calculated actually as the cross product of two of its edges? Basically the cross product of these two edges will then give as a face normal with length proportional to the area of the triangle. For free!

Ok, this means we must not normalize the face normals, and just accumulate them on the vertices so that we do a high quality vertex normal calculation. We just skipped one vector normalization per face (meaning, one square root and a division, or an inverse-square root)!

Looping only once

Some implementations make two passes on the mesh to normalize it, one on the faces to calc face normals and vertex-to-normal connectivity, and a second one where this information is used to do the actual normal accumulation for each vertex. This is unnecessary, we can make the code a lot smaller and faster, and use less memory by doing it all in one pass. For each face on the mesh, calc the face normal (without normalization, as just explained), and directly accumulate this normal in each vertex belonging to the face. After you are done with the faces, each vertex will have recieved all the face normals it was supposed to recieve. That simple.

The code

To finish, let's put all together in a small piece of code:

```
void Mesh_normalize( Mesh *myself )
{
    Vert      *vert = myself->vert;
    Triangle *face = myself->face;

    for( int i=0; i<myself->mNumVerts; i++ ) vert[i].normal = vec3(0.0f);

    for( int i=0; i<myself->mNumFaces; i++ )
    {
        const int ia = face[i].v[0];
        const int ib = face[i].v[1];
        const int ic = face[i].v[2];

        const vec3 e1 = vert[ia].pos - vert[ib].pos;
        const vec3 e2 = vert[ic].pos - vert[ib].pos;
        const vec3 no = cross( e1, e2 );

        vert[ia].normal += no;
        vert[ib].normal += no;
        vert[ic].normal += no;
    }
}
```

```
for( i=0; i<myself->mNumVerts; i++ ) verts[i].normal = normalize( verts[i].normal )  
}
```

This is quite fast, fast enough to do lot of mesh normalizations per frame. On top of it, if you are using vertex shaders you can be interested on skipping the last vertex normalization and do it on the shader (last line on the code above). Also in some cases, like 64 or even 4 kilobyte demos, it's usual to have all allocated buffers automatically initialized to zero. In that case, if this is the first and only normalization for a given mesh, you may skip the first loop on the function too of course.

inigo quilez - learning computer graphics since 1994
